Market-Aware LLM Inference Routing: You Cannot Pick a Provider From the Price List

Tony Wen
ts2015656@gmail.com

Abstract

Existing LLM routers treat each model's cost as a static per-model attribute and select a model by query difficulty. We argue this misses a first-order effect of the emerging, commoditized compute market: for open-weight models served by *competing providers*, the same model varies sharply in price, latency, reliability, and—silently—in answer *quality*. We formalize *market-aware routing* as a price-*taker* decision problem (the router optimizes against exogenous, provider-given prices, distinguishing it from provider-side pricing games) and show, on 18 (model×task) cells across 6 open models and competing providers measured live through a public aggregator, that price, latency, and quality are mutually uncorrelated: cross-provider price spreads reach 8.8×, latency spreads 68×, and "the same model" ranges from 35% to 90% accuracy on hard reasoning. Consequently neither price nor the advertised quantization label predicts quality, and "route to the cheapest" silently drops below a quality floor in 5 of our cells. A measurement-driven router that picks the cheapest provider which is *both* measured-equivalent *and* currently healthy realizes a median 50% cost reduction at matched quality. We release the measurement harness, the consolidated map, and an OpenAI-compatible routing gateway. We are explicit that our evidence is cross-sectional, not temporal; we discuss what the (not-yet-live) token spot market would add.

1 Introduction

LLM inference is being commoditized. GPU rental prices are published as daily indices and a GPU *compute futures* market was announced in 2026 [3]; the same open-weight model (Llama, Qwen, DeepSeek) is now served by dozens of competing providers at prices that differ by up to an order of magnitude. Yet the dominant line of LLM routing work —FrugalGPT [2], RouteLLM [7], Hybrid LLM [4], CARROT [9], MixLLM [10]— represents cost as a *static* per-model quantity (a published price, or the fraction of traffic sent to an expensive tier) and routes on query difficulty alone.

This paper makes a simple empirical claim with a sharp consequence: **across competing providers, price, latency, reliability, and quality of the same model are mutually uncorrelated, so a provider cannot be chosen from the price list—it must be measured.** The cheapest endpoint is typically the most aggressively quantized (fp8/fp4) and sometimes silently loses accuracy; but the most *expensive* endpoint is often neither the best nor reliable. Only direct, per-task measurement separates the safe-cheap provider from the cheap-and-degraded one.

Contributions. (i) We formalize market-aware routing as a *price-taker* constrained decision problem (Section 3), with a *price-taker assumption* that cleanly separates it from provider-side pricing games [1] and from infrastructure-layer spot serving [6, 5]. (ii) We measure 18 (model×task) cells across 6 open models and their competing providers live (Section 4), and show price$\perp$quality$\perp$latency$\perp$reliability (Section 5). (iii) We build a measurement-driven router and show a median 50% cost reduction at matched measured quality while avoiding the 5 "cheapest-is-degraded" traps (Section 6). We release the harness, the map, and an OpenAI-compatible gateway.

2 Related Work

Routing under static cost. The client-side routing lineage selects a model per query under a static cost model: a published per-token price or the offload fraction to a cheaper tier [2, 7, 4, 9, 10]. None model time-varying prices,

provider capacity, or congestion-dependent latency, and—central to our finding—none model *cross-provider* quality heterogeneity for a fixed model. CARROT's SPROUT benchmark is "price-aware" only in the sense of folding static list prices into the estimate; this is orthogonal to measuring a live market.

Provider-side pricing. A recent line studies how a *provider* should price its service. PriLLM [1] models the market as a Stackelberg game where the provider is the leader *setting* price to maximize profit. This is the *inverse* of our problem: we are a client/router that takes the market price as exogenous input. Price is the leader's decision variable there; it is our environment's state here.

Spot/preemptible serving. SpotServe [6] and SkyServe [5] cut cost by running inference on cheap preemptible GPUs and adapting to *availability*, assuming a *static* spot-vs-on-demand gap. They act on *where/how* a model is served, not *which* provider answers a query, and sit on the opposite side of the routing boundary. Reacting to exogenous time-varying prices is also established in carbon-/electricity-aware datacenter scheduling, but that acts on operator-controlled physical resources, not client-side cross-provider API routing.

3 Problem Formulation

Queries $x_1, \ldots, x_T$ arrive over a horizon. A pool $M = \{m_1, \ldots, m_K\}$ of model/provider endpoints each has a market state $(p_m(t), c_m(t), \ell_m(t), \phi_m(t))$: token price, available capacity, congestion-dependent latency, failure rate. Each query carries a task type, a quality requirement θ, a deadline d, and a value v.

Assumption 1 (price-taker / exogenous market). The market state evolves independently of the router's action: $P(m(t{+}1) \mid m(t), a_t) = P(m(t{+}1) \mid m(t))$. A single client's dispatch does not move provider prices or aggregate congestion. This one line is the formal separation from provider-side pricing [1], where price is endogenous.

Objective. With state $s_t = (x_t, m(t), b_t, \rho_t)$ (query, market, remaining budget, reserved inventory) and action a_t (provider, spot/reserved tier, defer, fallback), we maximize value-weighted quality subject to budget, SLO, and quality-floor constraints:

$$\max_{\pi} \ \mathbb{E}\Big[\textstyle\sum_t v_t \,\mathrm{score}(y_{a_t}, y^\star)\Big] \ \text{ s.t. } \ \mathbb{E}\big[\textstyle\sum_t \mathrm{cost}\big] \le B, \ \ \Pr(\ell_{a_t} > d_t) \le \delta, \ \ Q(x_t, a_t) \ge \theta_t.$$

Proposition 1 (bandit reduction). If there is no shared budget, no reserved inventory, the market is exogenous (Assumption 1), and no deferral, the constrained MDP decouples into independent per-query problems and the optimal policy is the greedy per-query utility maximizer $a_t^\star = \arg\max_m U_m(x_t, m(t))$; a contextual bandit over $(x_t, m(t))$ suffices. Shared budget, reserved inventory, or deferral break this and require a constrained policy. Our empirical study operates in the bandit regime; we treat the per-query utility as Q_m measured offline minus price and latency penalties at the current market state.

When does measurement matter? Let π_{static} route on the time-averaged price and π_{mkt} on the realized state. The advantage of measurement is zero only when the arg-max provider is unchanged by the realization, and otherwise grows with the *dispersion* of price/quality/latency across providers and the frequency with which the optimal provider flips. The next sections show that dispersion is large *right now*, even before any temporal price dynamics.

4 Measurement Methodology

We probe each endpoint through a public multi-provider aggregator (OpenRouter [8]), pinning each request to a single provider with fallbacks disabled, so a measurement reflects *that* provider's serving stack (quantization, kernels, sampling). For each (model, task) we run an objective probe set (n=15–20, exact-match scored) over all competing

Table 1: Measured dispersion per (model×task). $|P|$ = competing providers; Acc. range = min–max accuracy of *the same model*; Price×/Lat.× = max/min spread; Route price = price of the cheapest measured-equivalent healthy provider; Save = its reduction vs. the premium (most expensive) provider at matched quality. † marks cells where the cheapest provider falls below the quality floor.

Model	Task	$\|P\|$	Acc. range	Price×	Lat.×	Route price	Save
deepseek-chat-v3.1	classification	7	100%–100%	2.3	68	\$0.500	57%
	extraction	6	93%–100%	1.8	34	\$0.625	46%
	math	7	100%–100%	2.3	24	\$0.625	46%
gemma-3-27b-it	classification	5	100%–100%	2.5	4	\$0.120	61%
	extraction	5	100%–100%	2.5	8	\$0.120	61%
	math	4	53%–72% †	1.9	3	\$0.305	0%
llama-3.1-8b-instruct	classification	5	100%–100%	8.8	8	\$0.025	89%
	extraction	5	80%–93% †	8.8	10	\$0.035	84%
	math	5	56%–85% †	8.8	13	\$0.220	0%
llama-3.3-70b-instruct	classification	9	100%–100%	4.8	25	\$0.265	79%
	extraction	10	93%–100%	5.0	6	\$0.210	80%
	math	11	40%–75% †	6.1	56	\$0.265	79%
llama-4-maverick	classification	4	100%–100%	2.0	3	\$0.375	50%
	extraction	4	93%–93%	2.0	4	\$0.375	50%
	math	4	80%–85%	2.0	2	\$0.375	50%
mistral-small-3.2-24b-instruct	classification	4	100%–100%	1.5	2	\$0.172	14%
	extraction	4	93%–100% †	1.5	3	\$0.200	0%
	math	4	100%–100%	1.5	4	\$0.138	31%

providers and record accuracy, mean latency, real per-call cost, and availability (served fraction, with retry/back-off and error typing: 429/5xx/timeout/client). Tasks span a difficulty range: multi-step *math* (reasoning), *extraction*, and *sentiment classification*. Endpoints are deduplicated by (provider, quantization, price).

Honest measurement caveats. (a) Probe sets are small ($n \leq 20$): coarse distinctions (e.g. 50% vs. 55%) are within binomial noise, while the large gaps we emphasize (35% vs. 90%) are several standard errors. (b) Firing $\sim$20 rapid pinned requests at one provider can self-induce rate limits; we early-abort a provider after three consecutive failures and report it as low-availability, which may understate true uptime. (c) Reasoning models require a sufficient token budget; an initial truncation bug collapsed a strong model to 15% before we corrected it—an instructive reminder that measurement harnesses are themselves a source of error.

5 Empirical Findings

Table 1 reports all 18 cells. Three patterns hold.

(1) Quality dispersion is selective but large. On classification, providers saturate (all $\approx$100%) even for a weak 8B model. On hard multi-step math, "the same model" ranges 35–90% (8B), 40–75% (70B), 53–72% (others): aggressive quantization and serving choices silently cost tens of points. Dispersion is driven by *task difficulty*, not model size alone—the headline 35–90% gap is a weak-model×hard-task worst case.

(2) Price and latency dispersion are universal. Even on saturated tasks, price spreads reach 8.8× and latency spreads 68× for the identical model. The most expensive provider is frequently neither the most accurate nor the most reliable.

(3) Neither price nor quant label predicts quality. Two providers serving the same "fp8" model differ by 30 accuracy points; a bf16 endpoint can be the most expensive and among the least accurate. Only direct measurement orders providers correctly.

6 Market-Aware Router

We route each request to the cheapest provider whose measured accuracy is within a floor of the best *and* whose availability exceeds a threshold. Against a "premium" baseline (pay up for the dearest provider) the router achieves a median 50% cost reduction at matched measured quality; against a "price-blind" baseline (sort by price) it avoids the 5 cells where the cheapest provider is silently sub-floor. The same policy, exposed as an OpenAI-compatible endpoint, lets a caller change one line (`base_url`) and tag the task to obtain the per-task floor; a savings ledger records the counterfactual cost. On capable models where quality saturates, the router instead trades a little price for reliability, avoiding cheap-but-rate-limited endpoints.

7 Limitations

Our evidence is *cross-sectional*: it establishes large dispersion *at an instant*, not that the cheapest-safe provider *churns over time*. The latter—the temporal, "dynamic price" story motivated by compute futures [3]—requires a longitudinal trace we have not yet collected, and a live token spot market does not yet exist (token list prices change on the order of days). Our measurements are also point-in-time and provider availability is partly confounded by self-induced rate limits. We therefore frame the contribution as measurement-driven routing under *static-but-heterogeneous* markets, with the dynamic regime as motivated future work.

8 Conclusion

The same open model, served by competing providers, differs enough in price, latency, reliability, and silent quality that a provider cannot be chosen from the price list. Measuring the (model$\times$task$\times$provider) cell and routing to the cheapest measured-equivalent healthy endpoint yields a median 50% saving at matched quality. The measurement map is the durable asset; the router and gateway are thin layers on top.

References

[1] Anonymous. Pricing online LLM services with a data-calibrated stackelberg routing game. *arXiv:2511.09062*, 2025.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv:2305.05176*, 2023.

[3] CME Group. CME group and silicon data to launch first compute futures market, 2026. Press release, May 12, 2026.

[4] Dujian Ding, Ankur Mallick, Chi Wang, et al. Hybrid LLM: Cost-efficient and quality-aware query routing. In *ICLR*, 2024. arXiv:2404.14618.

[5] Ziming Liu et al. SkyServe: Serving AI models across regions and clouds with spot instances. In *EuroSys*, 2025. arXiv:2411.01438.

[6] Xupeng Miao et al. SpotServe: Serving generative large language models on preemptible instances. In *ASPLOS*, 2024. arXiv:2311.15566.

[7] Isaac Ong et al. RouteLLM: Learning to route LLMs with preference data, 2024. LMSYS; arXiv:2406.18665.

[8] OpenRouter. OpenRouter: A unified interface for LLMs, 2026. openrouter.ai.

[9] Seamus Somerstep et al. CARROT: A cost-aware rate-optimal router for LLMs. *arXiv:2502.03261*, 2025.

[10] Xinyuan Wang et al. MixLLM: Dynamic routing in mixed large language models. In *NAACL*, 2025. arXiv:2502.18482.